

Matrizes Esparsas em Formato CSR

Representação, reconstrução e cálculo de AA^T

Sumário

1	Motivação	2
2	Representação CSR	2
2.1	Primeiro exemplo	2
2.2	Como interpretar <code>row_ptr</code>	3
3	Convertendo uma matriz densa para CSR	3
4	Reconstruindo a matriz a partir do CSR	4
4.1	Como descobrir as dimensões?	4
5	Produto AA^T na representação densa	5
6	Ideia do produto diretamente em CSR	6
6.1	Exemplo intuitivo	6
7	Percurso com dois cursores	6
7.1	Primeiro caso: <code>col_index[p] < col_index[q]</code>	7
7.2	Segundo caso: <code>col_index[p] > col_index[q]</code>	7
7.3	Terceiro caso: <code>col_index[p] = col_index[q]</code>	7
8	Execução detalhada do produto em CSR	7
8.1	Cálculo de C_{00}	8
8.2	Cálculo de C_{01}	8
8.3	Cálculo de C_{02}	8
8.4	Demais produtos internos	9
9	Pseudocódigo do algoritmo	9
9.1	Observações sobre o algoritmo	10
10	Exercícios	11
11	Síntese	12

1 Motivação

Uma matriz é chamada de *esparsa* quando grande parte de seus elementos é igual a zero. Em uma representação densa, todos os elementos são armazenados, inclusive os zeros. Isso pode desperdiçar memória e operações computacionais.

O formato CSR (*Compressed Sparse Row*) armazena somente os elementos não nulos, suas colunas e as informações necessárias para identificar a linha à qual cada elemento pertence. Ele é especialmente conveniente para algoritmos que percorrem a matriz por linhas.

2 Representação CSR

Considere uma matriz A com n linhas e m colunas. Sua representação CSR utiliza três vetores principais:

Vetor	Descrição
<code>values</code>	Armazena os valores não nulos, percorrendo a matriz linha por linha.
<code>col_index</code>	Armazena a coluna correspondente a cada elemento de <code>values</code> .
<code>row_ptr</code>	Indica onde começa cada linha nos outros dois vetores. Possui $n + 1$ posições.

Neste material, linhas e colunas são numeradas a partir de zero.

2.1 Primeiro exemplo

Considere:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 5 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}.$$

Percorrendo a matriz linha por linha, encontramos os valores não nulos 5, 8 e 3. Portanto:

$$\text{values} = [5, 8, 3], \quad \text{col_index} = [0, 2, 1].$$

Os dois vetores devem ser lidos em conjunto:

Posição k	<code>values</code> [k]	<code>col_index</code> [k]
0	5	0
1	8	2
2	3	1

O vetor que separa as linhas é:

$$\text{row_ptr} = [0, 0, 2, 2, 3].$$

Os elementos da linha i ocupam as posições

$$\text{row_ptr}[i] \leq k < \text{row_ptr}[i + 1].$$

Assim:

Linha	Início	Fim	Posições utilizadas
0	0	0	nenhuma
1	0	2	0 e 1
2	2	2	nenhuma
3	2	3	2

O limite final não é incluído. Por exemplo, o intervalo $[0, 2)$ contém as posições 0 e 1.

2.2 Como interpretar row_ptr

Uma interpretação útil é:

$$\text{row_ptr}[i] = \text{quantidade de elementos não nulos existentes antes da linha } i.$$

Consequentemente:

$$\text{não nulos na linha } i = \text{row_ptr}[i + 1] - \text{row_ptr}[i]$$

e

$$\text{row_ptr}[n] = \text{nnz}(A).$$

Valores consecutivos iguais em `row_ptr` indicam uma linha vazia. Se $\text{row_ptr}[i] = \text{row_ptr}[i + 1]$, não existe nenhuma posição k pertencente à linha i .

3 Convertendo uma matriz densa para CSR

Considere:

$$B = \begin{bmatrix} 0 & 4 & 0 \\ 6 & 0 & 8 \\ 0 & 0 & 0 \\ 9 & 0 & 3 \end{bmatrix}.$$

Percorrendo as linhas:

- linha 0: valor 4, na coluna 1;
- linha 1: valores 6 e 8, nas colunas 0 e 2;
- linha 2: nenhum valor não nulo;

- linha 3: valores 9 e 3, nas colunas 0 e 2.

Logo:

```
values = [4, 6, 8, 9, 3],
col_index = [1, 0, 2, 0, 2],
row_ptr = [0, 1, 3, 3, 5].
```

4 Reconstruindo a matriz a partir do CSR

Para reconstruir a matriz, criamos inicialmente uma matriz de zeros. Em seguida, para cada linha i e para cada posição

$$k = \text{row_ptr}[i], \dots, \text{row_ptr}[i + 1] - 1,$$

fazemos:

$$A[i][\text{col_index}[k]] = \text{values}[k].$$

Considere:

```
values = [7, 2, 5, 9],    col_index = [2, 0, 3, 1],
row_ptr = [0, 1, 1, 3, 4],
```

e suponha que a matriz possua quatro colunas. Como `row_ptr` possui cinco posições, a matriz possui $5 - 1 = 4$ linhas.

- Linha 0: intervalo $[0, 1)$. Temos $A[0][2] = 7$.
- Linha 1: intervalo $[1, 1)$. A linha é vazia.
- Linha 2: intervalo $[1, 3)$. Temos $A[2][0] = 2$ e $A[2][3] = 5$.
- Linha 3: intervalo $[3, 4)$. Temos $A[3][1] = 9$.

Portanto:

$$A = \begin{bmatrix} 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 5 \\ 0 & 9 & 0 & 0 \end{bmatrix}.$$

4.1 Como descobrir as dimensões?

O número de linhas é determinado diretamente:

$$n = \text{tamanho}(\text{row_ptr}) - 1.$$

Entretanto, os três vetores não determinam sempre o número exato de colunas. Podemos concluir apenas que:

$$m \geq \max(\text{col_index}) + 1.$$

No exemplo, o maior índice de coluna é 3, portanto existem pelo menos quatro colunas. A matriz poderia, porém, possuir colunas adicionais inteiramente nulas ao final. Por esse motivo, uma representação CSR completa normalmente também guarda as dimensões n e m .

5 Produto AA^T na representação densa

Antes de realizar a operação diretamente em CSR, é importante entendê-la na representação densa. Considere:

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 0 & 4 & 5 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 4 \\ 2 & 0 & 5 \end{bmatrix}.$$

Se

$$C = AA^T,$$

então:

$$C_{ij} = \sum_{k=0}^{m-1} A_{ik}(A^T)_{kj} = \sum_{k=0}^{m-1} A_{ik}A_{jk}.$$

Portanto:

$$C_{ij} = \text{produto interno entre as linhas } i \text{ e } j \text{ de } A.$$

Por exemplo:

$$C_{00} = [1, 0, 2] \cdot [1, 0, 2] = 1^2 + 2^2 = 5,$$

$$C_{01} = [1, 0, 2] \cdot [0, 3, 0] = 0,$$

$$C_{02} = [1, 0, 2] \cdot [0, 4, 5] = 2 \cdot 5 = 10.$$

Calculando os demais elementos:

$$C = \begin{bmatrix} 5 & 0 & 10 \\ 0 & 9 & 12 \\ 10 & 12 & 41 \end{bmatrix}.$$

Observe que C é simétrica:

$$C_{ij} = C_{ji}.$$

Isso sempre ocorre porque:

$$(AA^T)^T = (A^T)^T A^T = AA^T.$$

6 Ideia do produto diretamente em CSR

Não é necessário construir explicitamente A^T . Para obter C_{ij} , podemos calcular o produto interno entre as linhas i e j de A .

Na representação densa:

$$C_{ij} = \sum_k A_{ik} A_{jk}.$$

Uma parcela só pode ser diferente de zero quando:

$$A_{ik} \neq 0 \quad \text{e} \quad A_{jk} \neq 0.$$

Logo, o algoritmo precisa encontrar as colunas nas quais as duas linhas possuem valores não nulos simultaneamente. Em termos de conjuntos, ele procura a interseção dos índices de coluna das duas linhas.

6.1 Exemplo intuitivo

Considere:

$$L_i = [7, 0, 0, 2, 0], \quad L_j = [0, 4, 0, 5, 0].$$

As colunas não nulas são:

$$\text{linha } i : \{0, 3\}, \quad \text{linha } j : \{1, 3\}.$$

A interseção é:

$$\{0, 3\} \cap \{1, 3\} = \{3\}.$$

Somente a coluna 3 contribui:

$$L_i \cdot L_j = 7 \cdot 0 + 0 \cdot 4 + 0 \cdot 0 + 2 \cdot 5 + 0 \cdot 0 = 10.$$

7 Percurso com dois cursores

Para calcular o produto interno de duas linhas em CSR, usamos:

- p : posição atual entre os elementos não nulos da linha i ;
- q : posição atual entre os elementos não nulos da linha j .

Não comparamos diretamente p e q . Comparamos:

$$\text{col_index}[p] \quad \text{e} \quad \text{col_index}[q].$$

O procedimento pressupõe que os índices de coluna estejam ordenados dentro de cada linha.

7.1 Primeiro caso: $\text{col_index}[p] < \text{col_index}[q]$

A coluna apontada por p é menor. Como a lista de colunas da linha j está ordenada e já alcançou uma coluna maior, o elemento apontado por p não encontrará correspondência. Avancamos apenas p :

$$p \leftarrow p + 1.$$

7.2 Segundo caso: $\text{col_index}[p] > \text{col_index}[q]$

Agora a coluna apontada por q ficou para trás e não encontrará correspondência. Avancamos apenas q :

$$q \leftarrow q + 1.$$

7.3 Terceiro caso: $\text{col_index}[p] = \text{col_index}[q]$

As duas linhas possuem valores não nulos na mesma coluna. Portanto:

$$\text{soma} \leftarrow \text{soma} + \text{values}[p]\text{values}[q].$$

Depois, avançamos ambos:

$$p \leftarrow p + 1, \quad q \leftarrow q + 1.$$

Os dois elementos são avançados porque ambos já foram utilizados. Em uma matriz, uma linha possui apenas uma entrada em cada coluna; assim, não há razão para comparar novamente nenhum deles.

Comparação	Significado	Atualização
$\text{col_index}[p] < \text{col_index}[q]$	coluna de p ficou para trás	avança p
$\text{col_index}[p] > \text{col_index}[q]$	coluna de q ficou para trás	avança q
$\text{col_index}[p] = \text{col_index}[q]$	coluna comum	multiplica e avança ambos

O menor avança; se forem iguais, ambos avançam.

8 Execução detalhada do produto em CSR

Para a matriz usada anteriormente:

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 0 & 4 & 5 \end{bmatrix},$$

temos:

$$\begin{aligned} \text{values} &= [1, 2, 3, 4, 5], \\ \text{col_index} &= [0, 2, 1, 1, 2], \end{aligned}$$

$$\text{row_ptr} = [0, 2, 3, 5].$$

Inicializamos a representação de C :

$$C_{\text{values}} = [], \quad C_{\text{col_index}} = [], \quad C_{\text{row_ptr}} = [0], \quad \text{cont} = 0.$$

8.1 Cálculo de C_{00}

As duas linhas são a linha 0. Os cursores começam em:

$$p = \text{row_ptr}[0] = 0, \quad q = \text{row_ptr}[0] = 0.$$

1. $\text{col_index}[0] = 0 = \text{col_index}[0]$. Logo:

$$\text{soma} = 1 \cdot 1 = 1.$$

Avançamos p e q .

2. $\text{col_index}[1] = 2 = \text{col_index}[1]$. Logo:

$$\text{soma} = 1 + 2 \cdot 2 = 5.$$

Avançamos ambos novamente.

Como $C_{00} = 5 \neq 0$, armazenamos:

$$C_{\text{values}} = [5], \quad C_{\text{col_index}} = [0], \quad \text{cont} = 1.$$

8.2 Cálculo de C_{01}

Agora:

$$p = \text{row_ptr}[0] = 0, \quad q = \text{row_ptr}[1] = 2.$$

1. $\text{col_index}[0] = 0 < \text{col_index}[2] = 1$: avançamos p .
2. $\text{col_index}[1] = 2 > \text{col_index}[2] = 1$: avançamos q .

A linha $j = 1$ terminou e nenhuma coluna comum foi encontrada. Portanto:

$$C_{01} = 0,$$

e nada é inserido nos vetores de C .

8.3 Cálculo de C_{02}

Inicialmente:

$$p = \text{row_ptr}[0] = 0, \quad q = \text{row_ptr}[2] = 3.$$

1. $\text{col_index}[0] = 0 < \text{col_index}[3] = 1$: avançamos p .

2. `col_index[1] = 2 > col_index[3] = 1`: avançamos q .
3. `col_index[1] = 2 = col_index[4]`: encontramos a coluna 2 em comum:

$$soma = values[1]values[4] = 2 \cdot 5 = 10.$$

Armazenamos:

$$C_{values} = [5, 10], \quad C_{col_index} = [0, 2], \quad cont = 2.$$

Terminamos a linha $i = 0$, então:

$$C_{row_ptr}[1] = 2.$$

Essa primeira linha de C é $[5, 0, 10]$.

8.4 Demais produtos internos

i	j	Colunas comuns	Produto interno	Armazenamento
1	0	nenhuma	0	não armazena
1	1	1	$3 \cdot 3 = 9$	$C_{11} = 9$
1	2	1	$3 \cdot 4 = 12$	$C_{12} = 12$
2	0	2	$5 \cdot 2 = 10$	$C_{20} = 10$
2	1	1	$4 \cdot 3 = 12$	$C_{21} = 12$
2	2	1 e 2	$4^2 + 5^2 = 41$	$C_{22} = 41$

Ao final:

$$C_{values} = [5, 10, 9, 12, 10, 12, 41],$$

$$C_{col_index} = [0, 2, 1, 2, 0, 1, 2],$$

$$C_{row_ptr} = [0, 2, 4, 7].$$

Esses vetores representam:

$$C = \begin{bmatrix} 5 & 0 & 10 \\ 0 & 9 & 12 \\ 10 & 12 & 41 \end{bmatrix}.$$

9 Pseudocódigo do algoritmo

O pseudocódigo a seguir recebe uma matriz A , com n linhas, armazenada em CSR, e devolve $C = AA^T$, também em CSR.

Algoritmo 1: Multiplicação $C \leftarrow AA^T$ diretamente em CSR**Entrada:** matriz A em CSR, com n linhas**Saída:** matriz $C = AA^T$ em CSR

```
 $C.values \leftarrow []$ 
 $C.col\_index \leftarrow []$ 
 $C.row\_ptr \leftarrow$  vetor de  $n + 1$  zeros
 $cont \leftarrow 0$ 
para  $i \leftarrow 0$  até  $n - 1$  faça
    se  $A.row\_ptr[i] = A.row\_ptr[i + 1]$  então
         $C.row\_ptr[i + 1] \leftarrow cont$ 
        continue
    fim se
    para  $j \leftarrow 0$  até  $n - 1$  faça
         $soma \leftarrow 0$ 
         $p \leftarrow A.row\_ptr[i]$ 
         $q \leftarrow A.row\_ptr[j]$ 
         $fim_i \leftarrow A.row\_ptr[i + 1]$ 
         $fim_j \leftarrow A.row\_ptr[j + 1]$ 
        enquanto  $p < fim_i$  e  $q < fim_j$  faça
            se  $A.col\_index[p] = A.col\_index[q]$  então
                 $soma \leftarrow soma + A.values[p] \cdot A.values[q]$ 
                 $p \leftarrow p + 1$ 
                 $q \leftarrow q + 1$ 
            senão se  $A.col\_index[p] < A.col\_index[q]$  então
                 $p \leftarrow p + 1$ 
            senão
                 $q \leftarrow q + 1$ 
            fim se
        fim enquanto
        se  $soma \neq 0$  então
            inserir  $soma$  ao final de  $C.values$ 
            inserir  $j$  ao final de  $C.col\_index$ 
             $cont \leftarrow cont + 1$ 
        fim se
    fim para
     $C.row\_ptr[i + 1] \leftarrow cont$ 
fim para
retorne  $C$ 
```

9.1 Observações sobre o algoritmo

1. O laço interno percorre as linhas j de A . Cada linha j corresponde à coluna j de A^T .
2. O laço **enquanto** termina quando uma das duas linhas acaba, pois não será possível encontrar uma nova coluna comum.
3. O valor armazenado em $C.col_index$ é j , porque o produto calculado é C_{ij} .
4. Ao terminar todas as comparações da linha i , o valor atual de $cont$ é colocado em $C.row_ptr[i + 1]$.

5. O teste correto para armazenar o resultado é $soma \neq 0$. Usar apenas $soma > 0$ descartaria resultados negativos.
6. Os índices de coluna devem estar ordenados dentro de cada linha do CSR para que a estratégia dos dois cursores funcione.
7. Como AA^T é simétrica, uma versão otimizada poderia calcular somente uma parte da matriz. Entretanto, isso exigiria cuidados adicionais para construir a representação CSR completa. O pseudocódigo apresentado prioriza a clareza.

10 Exercícios

Exercício 1

Converta a matriz para CSR:

$$D = \begin{bmatrix} 2 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \end{bmatrix}.$$

Determine `values`, `col_index` e `row_ptr`.

Exercício 2

Reconstrua a matriz representada por:

$$\begin{aligned} \text{values} &= [6, 2, 8, 4], & \text{col_index} &= [1, 3, 0, 3], \\ \text{row_ptr} &= [0, 2, 2, 3, 4], \end{aligned}$$

sabendo que ela possui cinco colunas.

Exercício 3

Considere:

$$E = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & 0 \\ 4 & 0 & 5 \end{bmatrix}.$$

- a) Converta E para CSR.
- b) Calcule EE^T na representação densa.
- c) Simule o percurso dos cursores no cálculo de C_{02} .
- d) Escreva a representação CSR do resultado.